Data Loading

Modulo 2 · Lezione 4



Agenda

Un solo file per tutta la serata: `working/loaders.py`. Le slide sono il filo: quando c'è da lavorare, una **slide di switch** vi manda al posto giusto — finita l'attività, si torna qui.
🎤 = il docente scrive, voi replicate · ⌨️ = scrivete voi (docente in affiancamento)

	Blocco	Dove si lavora	Tempo
📊	Quattro formati, un solo `Document`	slide	35′
📊	Setup: quattro cose cambiano da oggi	terminale	5′
🎤	**Live coding: i cinque pezzi del modulo**	`working/loaders.py`	45′
⌨️	**Esercizio: `.txt` e `.csv` sono vostri**	`working/loaders.py`	20′
📊	Reveal: una riga o tutto il file?	slide	5′
📊	I limiti, e il ponte al cleaning	slide	10′

> La chiave è **quella di Modulo 2 · Lezione 1 – Modulo 2 · Lezione 3**, nessuna registrazione nuova (persa? **alzate la mano**: c'è la riserva del docente). Setup passo-passo in `scripts/CONSEGNA.md`; tutti i comandi si lanciano **dalla cartella** `scripts/`. Un giro completo del file costa **~$0.002**.



1. Tanti formati, un solo Document



Il modello legge testo, non file

Un LLM non "apre" un PDF: riceve **testo** nel prompt e basta. Tutto il lavoro di stasera sta tra il file sul disco e quella stringa. Finora l'input lo scrivevamo a mano nel codice — i dati veri stanno nei file.

📄 Il file

PDF, DOCX, HTML, Markdown. Ognuno è un contenitore con le sue regole — non è "solo testo".

🔧 Il loader

Apre il contenitore, ne tira fuori il **testo** e si segna **da dove arriva**.

🤖 Il modello

Riceve quel testo nel prompt. Se l'estrazione è sporca o incompleta, **risponde peggio**.



Quattro contenitori, quattro problemi diversi

La stessa frase — «il fatturato 2025 è cresciuto del 12%» — può stare in quattro formati che non si assomigliano per niente. Stessa informazione, **tecnologie incompatibili** per tirarla fuori.

`PDF`

Non è testo lineare: è un **layout** di blocchi posizionati. Le pagine esistono davvero. A volte la pagina è solo un'immagine scansionata.

`DOCX`

Un archivio **zip di XML**. Il testo c'è, ma annegato in tag di formattazione, stili, revisioni.

`HTML`

Un **albero di tag** dove il contenuto utile è circondato da menu, banner, sidebar, footer.

`Markdown`

Il più gentile: è già testo. Ma la **sintassi** (#, *, tabelle, link) va gestita, non lasciata grezza nel prompt.



Perché serve un formato interno unico

Partiamo dal problema. Se ogni loader restituisse una forma diversa…

❌ Senza formato comune

Il loader PDF torna stringhe, l'HTML un albero, il DOCX una lista di paragrafi. **Ogni** stadio a valle — clean, chunk, embed — dovrebbe conoscerli tutti e gestirli uno per uno. La conoscenza del formato si **spande** in tutta la pipeline.

✅ Con un `Document` unico

Ogni loader normalizza nello **stesso** contenitore: testo + metadata. Da lì in poi nessuno sa più (né deve sapere) se veniva da un PDF o da una pagina web. Il formato si ferma **qui**.

È lo stesso modello su cui convergono tutte le librerie del settore: LangChain, LlamaIndex e Haystack hanno tutte un `Document` = testo + metadata.



Il `Document`: testo + metadata

Quattro righe. La teniamo magra di proposito: più è sottile, meno somiglia a un framework.

```python
from dataclasses import dataclass, field

@dataclass
class Document:
    text: str                                  # il contenuto estratto
    metadata: dict = field(default_factory=dict)  # da dove arriva
```

working/loaders.py

Perché una `dataclass` e non un `dict`

Type hint, autocomplete, e un posto naturale dove appendere validazione più avanti — a costo quasi zero.

È il pezzo che attraversa tutto

Ogni stadio del modulo riceve e produce `Document`. Lo scriviamo una volta, lo riusiamo fino al progetto.



I metadata servono alle lezioni dopo

Insieme al testo teniamo **da dove arriva**. Si raccolgono adesso, mentre carichiamo, gratis: dopo non si recuperano più.

`source`

Il file di origine. Servirà per le **citazioni**: «questa risposta viene da *relazione-annuale.pdf*».

→ Modulo 2 · Lezione 11

`page`

La pagina, sui PDF. Per puntare al punto esatto e per il **debug** del retrieval.

→ Modulo 2 · Lezione 8

`type`

Il formato d'origine. Servirà per **filtrare** le ricerche nel vector DB (solo PDF, solo pagine web…).

→ Modulo 2 · Lezione 9



2. I loader



Da oggi cambiano quattro cose

Rispetto a Modulo 2 · Lezione 3 il contesto di lavoro cambia. Ve le diciamo tutte e quattro adesso, così non le scoprite una alla volta.

`1 · dove sta il venv`

Si crea in `scripts/`, non più dentro `starter/`. Niente più `starter/`: tre cartelle con tre ruoli.

`2 · il toolkit ha un nome`

Due file sciolti andavano bene finché erano due. Da oggi il toolkit cresce a ogni lezione: gli diamo un nome, `aikit/`. Dentro, **gli stessi** `llm_client.py` e `pricing.py` della lezione scorsa, a meno di un import.

`3 · la riga strana in cima`

`sys.path.insert( ... )` è **già scritta** nel file: serve perché `working/` importa da `scaffolding/`, cartella **sorella**. L'editor sottolineerà l'import di `aikit`: **si sbaglia lui**, il programma gira.

`4 · quando è "fatto"`

Non più «check tutto `[OK]`». Da stasera è **quello che si vede a schermo**: si lancia il file e si guarda che cosa stampa.

```
scripts/
├─ scaffolding/ c'è già, non si tocca
├─ working/ qui si lavora
└─ dataset/ i documenti
```

```
cd scripts
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```



Una libreria per formato

Niente framework: per ogni formato una libreria **nativa**, quella che fa una cosa e la fa bene. Le incolliamo dietro la stessa interfaccia `Document`.

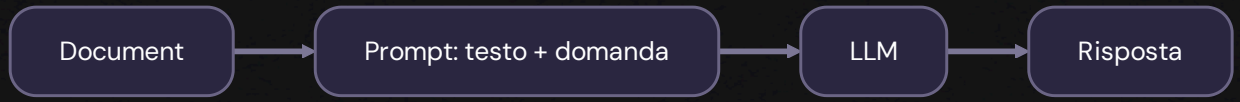
Formato	Libreria	Cosa fa	Dove fatica
PDF	`pypdf`	estrae il testo pagina per pagina	tabelle, colonne, scansioni
DOCX	`python-docx`	legge i paragrafi dall'XML	tabelle, caselle di testo
HTML	`BeautifulSoup`	naviga l'albero, estrae il testo	separare il contenuto dal contorno
Markdown	`markdown-it-py`	risolve la sintassi → testo	—

Nessuna di queste "capisce" il documento: lo **parsa**. Reggono bene il testo lineare; sui documenti complessi mostrano i limiti — ci torniamo alla fine.



Il banco di prova: la domanda al modello

Come sappiamo se un loader funziona *davvero*? Non basta vedere del testo a schermo: deve essere testo che il **modello capisce**. Quindi dopo ogni loader gli facciamo una domanda vera sul documento appena caricato.



`ask()` è **già scritta**, in `scaffolding/ask.py`: riusa il `LLMClient` della lezione scorsa e gira la domanda con `temperature=0` — un banco di prova che cambia idea a ogni lancio non serve a niente.

⚠️ `ask()` è **naive** di proposito: mette *tutto* il testo nel prompt, niente ricerca. È la **baseline**: funziona stasera, a fine lezione ne vediamo i limiti.

`load()`: la porta unica

Chi carica un documento non deve sapere quale libreria serve per quale file. Una funzione guarda l'estensione e **smista**. È la forma che il modulo avrà a fine serata.

```python
def load(path):
    ext = Path(path).suffix.lower()
    if ext == ".pdf":                return load_pdf(path)
    if ext == ".docx":               return load_docx(path)
    if ext in (".html", ".htm"):     return load_html(path)
    if ext in (".md", ".markdown"):  return load_markdown(path)
    raise ValueError(f"Formato non supportato: {ext}")
```

working/loaders.py

Aggiungere un formato vuol dire aggiungere **una riga qui e una funzione sopra**. Ed è esattamente quello che farete voi nell'esercizio.



🎤 Live coding — i cinque pezzi

🎤 **LIVE CODING — si guarda e si replica**

`working/loaders.py` · 45 minuti

si torna alle slide
sul terzo loader

TODO 1 · `load_pdf` — 3 pagine, 3 Document, il numero di pagina nei metadata

TODO 2 · `load_docx` — cambia il corpo della funzione, esce di nuovo un Document

TODO 3 · `load_html` — prima senza togliere il contorno, poi togliendolo

TODO 4 · `load_markdown` — via la sintassi, resta il testo

TODO 5 · `load()` — la porta unica: smista per estensione

Il main è a stadi

Ogni pezzo finito stampa il suo risultato, gli altri dicono «ancora da fare»: si **rilancia dopo ogni TODO**.

```
python working/loaders.py # sempre dalla cartella scripts/
```



Che cosa c’è dentro una pagina web

Il terzo loader merita una sosta. In una pagina di supporto il contenuto utile sta in `<main>` — intorno c'è tutto il resto, e **parla dello stesso argomento con numeri diversi**.

`<header>`
Promo: «reso esteso a **100 giorni**» · newsletter: «**garanzia 10 anni** sul telaio in omaggio» · cookie banner

`<nav>`
20 voci di menu, breadcrumb, banner: «**garanzia 10 anni sul telaio**»

`<main> — il contenuto`
Reso: **30 giorni** (14 se in saldo).
Garanzia legale **2 anni** + commerciale **5 anni** su motore e **struttura meccanica**, 2 anni sull'elettronica.
Spedizione gratis sopra **199 euro**.

`<aside>`
LumaDesk Mini: «reso **14 giorni**, garanzia **1 anno**» · LumaCare: «fino a **10 anni**»

`<footer>`
Tre colonne di link · «spedizione gratis sopra **49 euro**» · «reso gratuito entro **100 giorni**» · legale · `<script>` di analytics

Il contorno è il **56%** del testo che si estrae da questa pagina.



`load_html`: togliere il contorno

Sei tag via, e basta. Lo scriviamo **in due tempi**: prima senza il ciclo, poi con.

```python
NOISE = ["script", "style", "header", "nav", "aside", "footer"]

def load_html(path):
    with open(path, encoding="utf-8") as f:
        soup = BeautifulSoup(f, "html.parser")
    for tag in soup(NOISE):        # ← il ciclo che aggiungiamo dopo
        tag.decompose()
    testo = soup.get_text(separator="\n", strip=True)
    return [Document(testo, {"source": str(path), "type": "html"})]
```

working/loaders.py

🤖 La domanda del main è: **«quanto dura la garanzia sul telaio della scrivania?»** — la facciamo due volte, prima con il contorno dentro e poi senza.



La stessa domanda, due risposte

Stesso file, stessa domanda, stesso modello. Cambia solo il ciclo di `decompose()`.

	Caratteri nel prompt	«10 anni» nel prompt	Risposta del modello
senza `decompose()`	4.571	sì	**10 anni** — sbagliata
con `decompose()`	1.989	no	**5 anni** — giusta

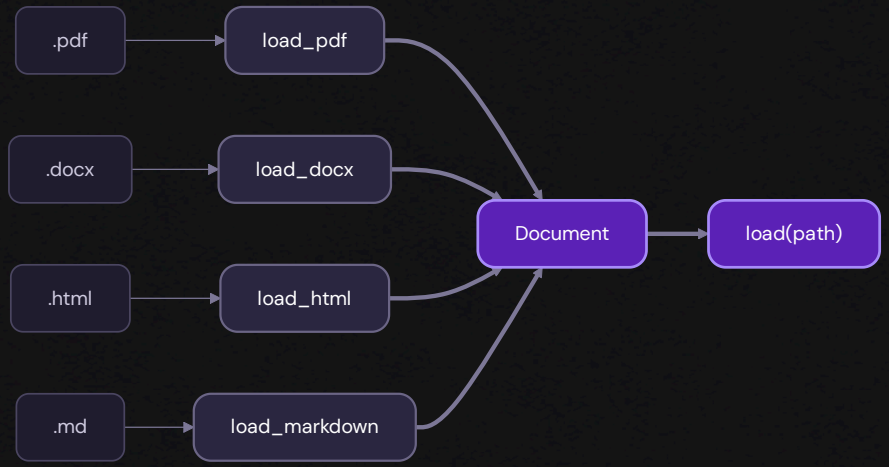
La risposta giusta è **5 anni**: `<main>` dice «5 anni sul motore e sulla **struttura meccanica**». La parola **«telaio» in** `<main>` **non c'è** — compare solo nel contorno, e lì accanto c'è «10 anni». Il modello ha scelto il pezzo di testo che somigliava di più alla domanda.

> È il modo tipico in cui un retrieval sporco sbaglia: **la risposta sbagliata usa le parole della domanda meglio di quella giusta**. Qui sono bastati sei tag ovvi — non basta sempre. «Pulire bene» è un mestiere a sé: **Modulo 2 · Lezione 5**.



Quello che avete scritto

Cinque pezzi, un file. Quattro loader diversi dietro la stessa interfaccia, e una porta sola per entrarci.




```
relazione-annuale.pdf → 3 Document · pdf
verbale-consiglio.docx → 1 Document · docx
faq-garanzia.html → 1 Document · html
guida-lumadesk.md → 1 Document · markdown
note-supporto.txt → ValueError: Formato non supportato: .txt
ordini.csv → ValueError: Formato non supportato: .csv
```

⌨ Esercizio — due formati sono vostri

⌨ **TOCCA A VOI — docente in affiancamento**

`working/loaders.py` · 20 minuti

si torna alle slide
per il reveal

Che cosa scrivete

`load_txt` e `load_csv`, e poi li **agganciate a** `load()`.
Prescritti solo i nomi, la firma e lo scopo: il come è vostro.
`dataset/note-supporto.txt` · `dataset/ordini.csv`

Fatto quando

Il main stampa i `Document` dei due file nuovi **e nessuna riga di** `load()` **dà più** `ValueError`.

Sul CSV c'è una scelta da fare: **una riga = un** `Document`, oppure **tutto il file in uno solo**? Non c'è una risposta giusta — ma pensate a che cosa ci farete dopo. Alla fine ne parliamo: chi ha scelto diversamente racconta perché.
Finito prima? Scrivete anche l'**altra** granularità e rilanciate: la risposta del modello cambia? Portate quello che avete visto al reveal.



Una riga o tutto il file?

Le due strade sui dodici ordini di `ordini.csv`. **Stasera danno lo stesso risultato**: `ask()` mette nel prompt tutti i Document comunque. La differenza si vede quando al posto di `ask()` c'è una **ricerca** che ne porta solo qualcuno.

`12 Document — uno per riga`

«Che cosa ha ordinato Nordvent?» → la ricerca porta **i suoi due ordini** e nient'altro.

«Quanti ordini sono consegnati?» → per contarli servono **tutte** le righe, e la ricerca ne porta poche.

`1 Document — tutto il file`

«Quanti ordini sono consegnati?» → arriva la tabella intera, il conto si fa.

«Che cosa ha ordinato Nordvent?» → arriva **tutto il file** anche per due righe.

> Nessuna delle due è sbagliata: **la granularità si sceglie in base alle domande** che ci si aspetta. È una decisione che torna, con un altro nome e su testi molto più lunghi, a **Modulo 2 · Lezione 10**.



3. I limiti



Tutto nel prompt non scala

`ask()` mette l'**intero** documento nel prompt. Stasera ha funzionato benissimo. Proviamo a immaginarlo con un PDF da trecento pagine, o con cinquecento documenti.

📄 I sei documenti di stasera

Ci stanno comodi nel **context window**, e il giro completo costa **$0,0023**. `ask()` funziona.

📚 Cinquecento documenti lunghi

Non ci stanno nel contesto (Modulo 1 · Lezione 10) — e anche dove ci stessero, si **ripagano a ogni singola domanda** (Modulo 2 · Lezione 2).

> Serve un modo per passare al modello **solo i pezzi di testo che contano** — ed è il resto del modulo.



Dal load al vector DB: le prossime lezioni

Caricare è il primo passo. Il muro che abbiamo appena toccato è la motivazione di tutto quello che viene dopo.



Spezzare i documenti, trasformarli in vettori, cercare i pezzi giusti e tenerli in un database fatto apposta. Non oggi — ma adesso sapete *perché* servono.

Quando il parsing nativo si rompe

C'è un secondo limite, sui documenti **difficili**. Le librerie di stasera reggono il testo lineare; su PDF con tabelle complesse, colonne o pagine scansionate vanno in crisi: celle alla rinfusa, testo a pezzi, pagine vuote.

🔧 Parsing locale (quello di stasera)

Gratis, privato, offline. Va benissimo per la maggior parte dei documenti. Si rompe sul **layout complesso**.

☁️ Parser gestiti (es. LlamaParse)

A pagamento, e i documenti vanno nel cloud. Ma **robusti** su tabelle e scansioni. Si agganciano come «un loader in più» dietro la stessa interfaccia.

🚩 Per oggi solo una bandierina: il confronto **locale contro gestito** lo facciamo con le mani al Lab (**Modulo 2 · Lezione 6**), su documenti veri e sporchi.



Il testo è caricato, non è pulito

Torniamo al primo loader. Il PDF ha dato tre `Document`, uno per pagina — e in cima a tutti e tre c'è la stessa cosa.

page 1 · 2.428 car

```
Lumen S.r.l. — Relazione annuale 2025
via Vittor Pisani 12 · P.IVA 09876543210
pagina 1 di 3
```

Relazione annuale 2025 · 1. Sintesi dell'esercizio…

page 2 · 2.743 car

```
Lumen S.r.l. — Relazione annuale 2025
via Vittor Pisani 12 · P.IVA 09876543210
pagina 2 di 3
```

I reclami in garanzia hanno riguardato l'1,8%…

page 3 · 1.222 car

```
Lumen S.r.l. — Relazione annuale 2025
via Vittor Pisani 12 · P.IVA 09876543210
pagina 3 di 3
```

8. Rischi e incertezze. Il rischio di concentrazione…

> Su tre pagine sono tre copie della stessa riga. Su un documento da trecento pagine sono **trecento copie**, tutte dentro il prompt. Prossimo passo: ripulirlo — **Modulo 2 · Lezione 5, Text Cleaning**.



Grazie

Prossima lezione: Text Cleaning & Pre-processing (Modulo 2 · Lezione 5)

